



SIMATS ENGINEERING

Descriptive Written Exam

Course Name: Software Testing

Course Code: CSA37

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3

10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3

17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Question 1: Redesigning Ad-hoc Testing in a Banking Application as an Engineering Activity

Introduction

Testing is not a last-minute activity of "trying the software"; it is an engineering discipline with planning, design, execution, measurement and improvement. The banking team fails frequently because tests are unplanned, undocumented and dependent on individual memory. The redesign introduces the same rigour that is applied to building the software.

Redesigned Approach

- **Test planning:** Define scope (fund transfer, balance enquiry, bill payment), risks, entry/exit criteria, schedule and responsibilities in a test plan.
- **Systematic test design:** Derive test cases from requirements using defined techniques instead of intuition, and maintain traceability from each requirement to its test cases.
- **Tools:** Use a test management tool (TestLink/Jira-Zephyr) for cases and results, a defect tracker (Jira/Bugzilla), Selenium/Appium for automated regression, and JMeter for load.
- **Standards:** Adopt IEEE 829 templates for test documents, IEEE 1044 for defect classification, ISO/IEC 25010 quality attributes and a common defect severity scale.
- **Measurement:** Track test coverage, defect density, defect detection percentage and pass rate to judge quality objectively.

Techniques to Replace Ad-hoc Testing

Equivalence partitioning, boundary value analysis, decision tables, state transition testing, use-case based testing and risk-based prioritisation replace random checking. Reviews and walkthroughs of requirements are added to find defects before coding.

How This Improves Defect Detection and Quality

Because every requirement has designed test cases, no feature is skipped; techniques target the places where defects usually hide; fixed defects are retested and regression-tested; and metrics show weak areas early. Quality becomes measurable and repeatable rather than accidental.

Practical Example

Requirement: a customer may transfer Rs. 100 to Rs. 50,000 per transaction. Ad-hoc testers might try only Rs. 5,000. Using boundary value analysis, the designed cases are 99, 100, 101, 49,999, 50,000 and 50,001, plus a transfer exceeding the account balance and a transfer to an

invalid account. A defect where Rs. 50,000 is wrongly rejected is found immediately, logged with steps and severity, fixed and added to the automated regression suite.

Conclusion

Treating testing as an engineering activity, supported by planning, techniques, tools and standards, converts unstructured testing into a controlled process that detects more defects earlier and reduces production failures.

Question 2: Structured Testing Process Across SDLC Phases for an ATM System

Introduction

Inconsistent ATM results occur because testing is performed differently each time. A defined process, mapped to each SDLC phase (the V-model approach), makes testing planned, repeatable and traceable. Each development phase is paired with a testing activity.

Stages and Their Relevance

- **Requirement analysis:** Testers review requirements for clarity and testability and prepare the test strategy. Ambiguities such as "daily limit" are clarified here, which is the cheapest time to fix them.
- **Design:** Test plan and test cases are designed from the design documents; the test environment and data are planned. This ensures every design element has verification.
- **Coding:** Unit testing and code reviews are performed by developers to catch logic faults early.
- **Integration:** Interfaces between the ATM, card reader, bank server and cash dispenser are tested to expose data-passing defects.
- **System testing:** The complete ATM is tested against requirements for function, performance, security and reliability.
- **Acceptance testing:** The bank validates that the system meets business needs before release.
- **Maintenance:** Regression testing confirms that fixes and changes do not break existing behaviour.

Application to ATM Withdrawal

Requirement analysis: confirm rules (multiples of Rs. 100, maximum Rs. 20,000 per day, minimum balance). Design: write cases for valid withdrawal, amount above balance, amount above daily limit, non-multiple amount, wrong PIN three times, card blocked, cash dispenser empty, and network failure after debit. Unit and integration: verify the balance-check and debit modules, then the bank-server interface. System: execute all cases with recorded

expected and actual results. Acceptance: bank users perform real scenarios. Maintenance: re-run the withdrawal suite after every change.

How Process Consistency Reduces Defects

A fixed process means the same entry and exit criteria, templates and defect handling are used every cycle. Nothing depends on an individual tester, defects are found in the phase in which they arise (reducing rework cost), and results become comparable between releases.

Expected Outcomes

Consistent and reproducible results, fewer withdrawal failures such as double debit or wrong cash dispensed, early defect detection, complete documentation and traceability, lower cost of fixing defects, and higher customer confidence.

Question 3: Errors, Defects and Failures in a Web Application

Introduction

Following the standard (IEEE/ISTQB) terminology, an error (mistake) is a human action that produces an incorrect result; a defect (fault, bug) is the flaw in the code or document caused by the error; a failure is the deviation of actual behaviour from expected behaviour seen when the defective code executes.

Relationship

Error leads to defect, and defect, when executed, may lead to failure. Not every defect causes a failure, because the faulty code may never run under the given inputs. A failure can also result from the environment, not only from a defect.

Application to the Login and Transaction Scenario

- **Error:** The developer misreads the requirement and writes the password check as case-insensitive.
- **Defect:** The code line compares passwords using a case-insensitive function.
- **Failure:** A user logging in with "Pass@123" is also accepted with "pass@123", whereas the expected result is rejection.
- **Second example:** A developer forgets that the transfer must deduct the amount from the sender (error); the debit statement is missing (defect); the sender's balance remains unchanged after a successful transfer (failure).

Effect of Improper Understanding

If the terms are confused, testers report symptoms as causes, developers cannot locate the faulty code, duplicate or vague reports are raised, severity is misjudged, and root-cause

analysis and prevention are not possible. Management also gets wrong defect statistics, since failures and defects are counted as the same thing.

Improving Defect Reporting Clarity

- Use a standard template: ID, title, module, environment, steps to reproduce, expected result, actual result, severity, priority, screenshot/logs and status.
- Report the observed failure factually (actual versus expected), and leave the diagnosis of the defect to the developer.
- Write one defect per report and make steps repeatable by another person.

Example of a Clear Report

Title: Login accepts password with wrong letter case. Steps: register with Pass@123; log out; log in with pass@123. Expected: login rejected with an error message. Actual: user logged in. Severity: High (security).

Conclusion

A clear understanding of error, defect and failure gives precise defect reports, faster debugging and reliable quality measurement.

Question 4: Applying Software Testing Principles to Defect-Concentrated Modules

Introduction

When most defects are found in a few modules, testing effort must be directed by principles rather than spread equally. The seven testing principles provide this guidance.

Principles and Their Role in Test Planning

- **Testing shows presence of defects:** Passing tests do not prove the absence of defects, so planning must be risk-focused and the remaining risk communicated.
- **Exhaustive testing is impossible:** All input combinations cannot be tested, so cases are selected by risk and priority.
- **Early testing:** Reviews and test design start with requirements, so defects do not reach the problem modules.
- **Defect clustering:** A small number of modules (often about 20 percent) contain most defects (about 80 percent, the Pareto principle). Planning allocates more testers, time and test depth to these modules.
- **Pesticide paradox:** Repeating the same tests stops finding new defects, so test cases must be reviewed and updated regularly.
- **Testing is context dependent:** A banking module needs stricter security and accuracy testing than a content page.

- **Absence-of-errors fallacy:** A defect-free system is useless if it does not meet user needs, so requirement validation remains essential.

Prioritising Test Cases

Rank cases by risk = likelihood of failure x business impact. Use defect history to place cases for clustered modules in the first group (high priority), core business flows in the next group, and cosmetic or rarely used features last. Within the high-priority modules, apply boundary, negative and integration cases, and run them first and in every regression cycle.

Improvement in Testing Efficiency

Effort is focused where defects are likely, so more defects are found in less time, the schedule delay is reduced, regression suites are smaller but more effective, and developers can concentrate code review and refactoring on the weak modules.

Real-time Example

In an online shopping project, defect logs showed that the Payment and Cart modules contained 70 percent of defects. The team assigned two additional testers, added boundary and failure-recovery cases (card declined, timeout, coupon combinations), executed them first each build, and refreshed old cases to avoid the pesticide paradox. Defect leakage to production dropped and the release was completed on time.

Conclusion

Applying defect clustering and the other principles makes test planning risk-based, efficient and data-driven.

Question 5: Defect Classification and Repository to Prevent Recurring Defects

Introduction

A defect repository is a central database of all defects with their attributes. Without analysis it is only storage; with classification it becomes a source of prevention.

How Defects Originate

Defects arise from human errors at every phase: unclear or incomplete requirements, wrong design decisions, coding mistakes, incorrect data or configuration, weak or missing testing, and changes made without impact analysis. Time pressure, lack of skills and poor communication increase these errors.

Categories of Defects

- **By origin/phase:** requirement, design, coding, testing and deployment defects.
- **By type:** functional, logical, interface, data, performance, security and usability defects.
- **By severity:** critical, major, minor and trivial.

- **By priority and status:** high/medium/low; new, assigned, fixed, retested, closed or reopened.
- **IEEE 1044 style classes:** specification, design, code, documentation and environment.

Using Repository Data for Test Design and Prevention

Analysis of the repository shows which modules, defect types and phases produce most defects. Test cases are then designed for recurring types (for example boundary defects), regression suites include every previously found defect, and modules with high defect density receive deeper testing. For prevention, root-cause analysis of frequent classes leads to coding checklists, review focus and training. For example, if 40 percent of defects are input-validation defects, a validation checklist and boundary test set are added to every release.

Suggested Improvements

- Mandatory fields for module, type, phase, severity and root cause in every defect record.
- Periodic analysis with Pareto charts and defect trend reports for each release.
- Add a regression test for each fixed defect and review lessons learned after every release.
- Link defects to requirements and test cases to ensure traceability.

Expected Quality Improvements

Fewer repeated defects, lower defect density, higher defect detection effectiveness, shorter fixing time, better release stability and a continuously improving process.

Question 6: Root Cause Analysis of Crashes in a Mobile Banking Fund Transfer

Introduction

Crashes during fund transfer indicate failures caused by defects that were not detected in testing. Testing principles such as early testing, defect clustering and exhaustive testing being impossible guide a systematic investigation.

Identifying the Root Cause

- Reproduce and record the failure with exact steps, device, OS version, network type and data.
- Collect crash logs, stack traces and server logs to locate the failing module.
- Classify the failure: input handling, null reference, memory, network timeout, concurrency, or API mismatch.

- Use the 5-Whys or fishbone method to move from the failure to the defect and then to the error in the process that allowed it.

Likely Causes with Examples

A null value when the beneficiary list is empty; an unhandled exception when the network drops after the debit request; memory overflow on low-end devices; an unvalidated amount field accepting symbols or very large numbers; two simultaneous taps submitting duplicate requests; a mismatch between app and server API versions.

How Systematic Testing Reduces Such Issues

Designed test cases covering valid, invalid and boundary inputs, interrupted network, low battery or memory, and multiple devices find these faults before release. Risk-based priority makes fund transfer a top-priority area, and regression testing prevents reintroduction.

Illustration

Test case: transfer Rs. 1,000 and switch off the network after tapping Confirm. Expected: a clear message, no duplicate debit and the transaction status verifiable. Actual: the app crashes and the amount is debited. This reveals missing exception handling and a missing transaction rollback.

Improvements in Testing Practices

- Add negative, boundary, interruption and device-compatibility testing on real devices and emulators.
- Perform load, stress and security tests on the transfer API.
- Automate regression using Appium, and monitor crashes with tools such as Firebase Crashlytics.
- Perform code reviews and static analysis for exception handling, and conduct root-cause review after each crash.

Conclusion

Careful root-cause analysis and systematic, risk-based testing significantly reduce crashes and protect customer money and trust.

Question 7: Testing versus Debugging

Introduction

Testing and debugging are related but different activities. Confusing them leads to defects being found late, wrongly fixed or not found at all.

Differentiation

- **Purpose:** Testing aims to detect failures and defects by executing the software; debugging aims to find the cause of a known failure and fix it.
- **Performed by:** Testing by testers (and developers at unit level); debugging by developers.
- **Nature:** Testing is planned, can be scheduled and can be automated; debugging is investigative and cannot be fully scheduled.
- **Input and output:** Testing starts with test cases and produces defect reports and results; debugging starts with a defect report and produces a corrected code.
- **Knowledge:** Testing can be done without knowing the code (black box); debugging requires detailed knowledge of the code.

Roles in Software Quality

Testing gives information on quality and shows risk; debugging removes the defect and improves quality. Quality needs both: a defect that is not found cannot be fixed, and a defect that is found but not corrected remains in the product. Retesting after debugging confirms the fix.

Effect of Improper Usage

If developers only debug the failures they happen to notice, they do not perform planned testing, so many defects escape. If testers try to fix code, the independence of testing is lost. Treating a quick patch as testing leads to symptoms being fixed and root causes remaining, new defects appearing without regression tests, and unreliable reports. Example: a developer fixes a wrong interest figure for one account without testing other account types, so the same defect exists elsewhere.

Structured Integration Approach

- Tester executes planned cases and logs a complete defect report.
- Developer reproduces, locates the cause using debugging tools and corrects the code.
- Tester performs confirmation testing of the fix.
- Regression tests are run to ensure no side effects.
- The defect is closed and added to the repository for analysis.

Conclusion

Keeping the roles separate but linked through a defect life cycle makes defect resolution effective and improves software quality.

Question 8: Cross-Browser Testing Strategy for an E-commerce Platform

Introduction

Inconsistent behaviour across browsers shows that the platform was tested on a limited set of browsers. Principles such as exhaustive testing being impossible, risk-based testing and defect clustering guide an efficient cross-browser strategy.

Strategy

- **Select the browser matrix:** Use analytics to identify the most used browsers, versions, operating systems and devices (Chrome, Firefox, Safari, Edge, mobile browsers) and prioritise them.
- **Define scope:** Test critical flows first (search, cart, checkout, payment, login), then layout and secondary features.
- **Design test cases:** Include functional, layout/responsiveness, JavaScript behaviour, form validation, cookies and performance.
- **Execute and compare:** Run the same cases on every browser and compare actual results with the baseline.
- **Report and retest:** Log browser-specific defects with browser name and version and run regression tests after fixes.

How Defects Originate

Browsers interpret HTML, CSS and JavaScript differently; features or APIs are supported in some versions and not others; vendor-specific rendering engines, outdated browsers, missing polyfills, hard-coded styles, and different handling of cookies, fonts and security settings all create defects. The root error is the developer's assumption that one browser represents all.

Illustration of Improved Reliability

Example: the Pay Now button works in Chrome but is hidden in Safari because of an unsupported CSS property. A systematic matrix test detects this before release, the style is corrected with a compatible alternative, and the case is added to regression. Users on all browsers then see the same behaviour, increasing reliability and conversion.

Tools and Techniques

- Selenium WebDriver and Selenium Grid for automated parallel runs.
- BrowserStack, LambdaTest or Sauce Labs for real browser and device clouds.
- Playwright or Cypress for modern browser automation and visual comparison.
- Techniques: pairwise (combinatorial) testing of browser-OS-device combinations, responsive testing, visual regression testing and progressive enhancement.

Conclusion

A prioritised browser matrix, automated execution and structured defect reporting provide wide coverage with limited effort and consistent behaviour across browsers.

Question 9: Defect Classification for a Payroll System with Logical Errors

Introduction

Defect classification groups defects by common characteristics so that causes can be understood and prevented. In the payroll system the defects are mainly logical (algorithmic) errors in the code.

Categorisation of the Payroll Defects

- **By type:** Logic/algorithm defects (wrong formula for overtime), computation defects (rounding errors), data defects (wrong tax slab table), and control-flow defects (wrong condition in the leave-deduction loop).
- **By origin:** Mostly coding defects, but some may come from misinterpreted requirements or design.
- **By severity:** Critical, since employees receive wrong salary and there are legal and financial effects.
- **By detection phase:** Defects found in testing are cheaper than those found after salary processing.
- **Example:** Net salary = Basic + HRA + Allowances - Deductions is coded as Basic + HRA - Allowances - Deductions; the logic is wrong and every employee's salary is affected.

How Understanding Defect Classes Improves Testing

Knowing that logic defects dominate leads to the use of boundary value analysis, decision tables for tax slabs and allowances, white-box techniques such as decision and condition coverage, and independent recalculation of expected salaries. Test effort is prioritised on calculation modules, and test cases are tailored to each class.

Prevention in Future Releases

Peer code reviews and walkthroughs of formulas; unit tests with known expected salaries; validation of formulas by finance experts; checklists for common logic faults; static analysis tools; and root-cause analysis with the repository to track recurring defect classes.

Improvements in Testing Strategy

- Create decision tables for tax, bonus and deduction rules.
- Maintain a verified set of test data with manually computed expected results.

- Automate regression of salary calculation across employee categories and month-end boundaries.
- Add parallel testing against the old system's output and review defect trends each release.

Conclusion

Classifying the payroll defects directs testing at the right areas and supports preventive actions, producing correct and trustworthy salary calculation.

Question 10: Importance of Structured Test Case Design

Introduction

Executing random tests gives no assurance of coverage, repeatability or traceability. Applying testing as an engineering activity requires that tests be planned and designed before execution.

Redesigned Process

- Analyse requirements and identify test conditions.
- Select suitable design techniques and write test cases with ID, objective, preconditions, input data, steps, expected result and priority.
- Review the test cases and map them to requirements (traceability matrix).
- Execute, record actual results, log defects and measure coverage.

Importance of Structured Test Case Design

It ensures complete coverage of requirements, reduces redundant tests, makes tests repeatable by any tester, allows reuse in regression, helps estimate effort and progress, and gives evidence of testing. It also finds defects early when test design reveals ambiguities in requirements.

How It Improves Defect Detection

Techniques target defect-prone situations: boundary values, invalid classes, state changes and combinations that random testing rarely reaches. Example: for an age field accepting 18 to 60, random tests might use 25 and 40 and pass. Designed tests use 17, 18, 19, 59, 60, 61, blank and text, and detect that 60 is wrongly rejected.

Techniques to Enhance Test Effectiveness

- Black box: equivalence partitioning, boundary value analysis, decision table, state transition and use-case testing.
- White box: statement, branch and condition coverage.

- Experience based: error guessing and exploratory testing used in addition, with time-boxed charters.
- Peer review of test cases, risk-based prioritisation and periodic update to avoid the pesticide paradox.

Conclusion

Structured test case design replaces luck with method, giving higher coverage, better defect detection and more reliable quality.

Question 11: Data Inconsistency in a Healthcare Application

Introduction

Data inconsistency across modules (for example, a patient's allergy shown in registration but not in prescription) is a serious quality and safety risk. Testing principles such as early testing, risk-based testing and context dependence guide the analysis.

Analysis of the Issue

Identify which data items differ, in which modules and at what step (entry, storage, transfer or display). Trace the data flow between modules (registration, billing, pharmacy, laboratory) and compare database values with screen values.

How Defects Originate in Development

- **Requirement gaps:** Data definitions and formats are not standardised across modules.
- **Design defects:** Duplicate storage of the same data without synchronisation, or poor database normalisation.
- **Coding defects:** Different validation rules, data type or length mismatch, missing update of one module after a change, and wrong joins or queries.
- **Integration defects:** Interface or message mapping errors and failure of transactions part-way (no rollback).
- **Change defects:** Modification in one module without impact analysis.

Ensuring Data Integrity through Systematic Testing

Use end-to-end test scenarios, such as registering a patient, adding allergy, prescribing medicine and billing, and verify that the same data is correct at every point. Apply data validation, boundary and referential-integrity tests, database checks with SQL, concurrency tests, and security and audit-log tests. Example: update a patient's phone number in one module and verify the other three modules show the updated value.

Improvements in Defect Tracking and Reporting

- Record module, data item, source and target values, and screenshots or queries in each report.
- Classify defects (data, interface, logic) and track by severity, giving high priority to patient-safety issues.
- Link defects to requirements and test cases and review defect trends by module.
- Hold periodic defect review meetings among module owners.

Conclusion

Systematic, end-to-end testing together with disciplined defect tracking maintains reliable and consistent healthcare data.

Question 12: Using a Defect Repository to Eliminate Repeated Defects

Introduction

Repeated defects in spite of testing show that lessons from earlier defects are not being used. The defect repository, a structured record of all defects, can reveal why.

Analysis of the Issue

Possible reasons are that fixes address symptoms and not root causes, regression tests were not added for fixed defects, the repository has incomplete or inconsistent information, and defect data is not reviewed between releases. Analysis starts by grouping the repeated defects by module, type, cause and release.

How Repository Data Supports Test Design

- Defect-prone modules receive more test cases and deeper techniques.
- Each earlier defect becomes a regression test case.
- Frequent defect types (for example, validation) lead to dedicated checklists and techniques such as boundary analysis.
- Defects found late indicate gaps in earlier reviews and test levels.

Identifying Defect Patterns

Use Pareto charts, trend charts and defect density per module, plus root-cause categories. Example: the repository shows that 35 percent of defects across three releases were date-format errors, concentrated in Billing and Reports. This pattern shows a missing common date-handling standard.

Improvements to Prevent Recurring Defects

- Perform root-cause analysis for every major defect and record it.
- Add regression tests and update checklists for each fixed defect.

- Create coding and design standards for the recurring problems, with a shared date/format library.
- Review repository metrics at the end of every release and train the team on frequent defect types.
- Standardise defect fields to keep data reliable.

Conclusion

When the defect repository is analysed and used to guide test design and prevention, repeated defects fall and release quality steadily improves.

Question 13: Testing Gaps in a Web-based Learning System During Peak Usage

Introduction

Failures at peak usage show that non-functional scenarios, particularly load and stress, were not tested. Exhaustive testing is impossible, so risk-based planning must include expected peak conditions.

Identifying Gaps

- Only functional testing was done with a single user.
- No performance, load, stress or scalability testing for events such as examinations or result publication.
- Test data and environment were not similar to production.
- No monitoring of response time, database connections, memory or CPU.
- Requirements did not state the number of concurrent users.

Impact of Test Planning and Coverage on Performance

A test plan that lists performance objectives (for example 5,000 concurrent users with response time under 3 seconds), the environment, data and workload model ensures such scenarios are tested. Coverage of critical flows (login, video streaming, quiz submission, upload) under load exposes bottlenecks such as slow queries, thread exhaustion and memory leaks before release.

Better Strategy and Reliability

Example: simulate 5,000 students submitting an online quiz at the same time. The test shows that response time rises to 20 seconds and the database connection pool is exhausted. After tuning the pool, adding caching and indexing, the retest shows 2.5 seconds, so the system stays reliable during real peak usage.

Enhancements to Avoid Such Failures

- Include load, stress, endurance and spike testing using JMeter, Gatling or LoadRunner.

- Use production-like environments and realistic data, and define performance acceptance criteria.
- Monitor servers with tools such as Grafana or New Relic, and perform capacity planning.
- Run performance regression in each release and test failure recovery.

Conclusion

Planning for peak scenarios and covering non-functional requirements in the test strategy improves reliability and prevents failures during heavy usage.

Question 14: Role of the Tester in Early SDLC Stages

Introduction

When testers join only after coding, defects originating in requirements and design are found late and cost much more to fix. The principle of early testing says that testing activities should start as early as possible.

Applying the Tester's Role in SDLC

- **Requirements:** Review requirements for completeness, clarity, consistency and testability; raise questions.
- **Design:** Review design documents, identify risks, and prepare the test strategy, plan and test cases.
- **Coding:** Prepare test data and environment, support unit and integration testing and review test results.
- **Testing and release:** Execute tests, report defects, retest and give quality status.
- **Maintenance:** Perform regression and release-readiness testing.

Impact of Early Involvement on Quality

Ambiguities are removed before design, requirement and design defects are found by review rather than failure, test cases are ready when the build arrives, rework and cost are reduced, schedule risk is lower, and developers and testers share a common understanding of quality.

Practical Example

A requirement says "the system should be fast and allow reasonable failed logins". An early tester asks for measurable figures and the team defines response time under 2 seconds and account lock after 3 failed attempts. Without this, the defect would have been found during system testing, after the login module was coded, needing redesign. Early review saved time and cost.

Changes in Team Collaboration Practices

- Include testers in requirement and design review meetings and sprint planning.
- Adopt shift-left practices, acceptance criteria and three-amigos discussions (business, developer, tester).
- Share defect and test information through a common tool and daily stand-ups.
- Encourage mutual respect, joint ownership of quality and continuous feedback.

Conclusion

Early tester involvement prevents defects, lowers cost and improves quality through collaboration.

Question 15: Improving Test Coverage for Inconsistent System Output

Introduction

A system that works for some inputs and fails for others shows that the tested inputs and paths represent only a small part of the possible behaviour. Coverage measures how much of the requirements or code has been exercised by tests.

Identifying the Gaps

- Compare requirements and test cases through a traceability matrix to find untested requirements.
- Measure code coverage (statement, branch, condition) using tools such as JaCoCo or Coverage.py to find unexecuted code.
- Analyse failed cases to find the missing input classes, for example negative values, empty input or boundary values.
- Check which scenarios (alternate and error paths) are missing.

How Proper Coverage Ensures Reliability

When requirements, input classes, boundaries, decisions and error paths are covered, the chance of undetected defects falls and behaviour is verified for all important situations, not only the common one. Coverage also gives objective evidence about the completeness of testing, although 100 percent coverage still does not prove the absence of defects.

Improving Test Case Design

Example: a discount function gives 10 percent for orders of 1,000 or more. Original tests used 500 and 2,000. Improved design using equivalence partitioning and boundaries adds 0, 999, 1,000, 1,001, a negative number and a non-numeric value. The defect that exactly 1,000 gets no discount is then found. Additional branch tests verify each condition in the code.

Techniques for Better Validation

- Equivalence partitioning, boundary value analysis, decision tables and state transition testing.
- Statement, branch and condition coverage for white-box testing.
- Pairwise testing for combinations of inputs, error guessing and exploratory testing.
- Reviews of test cases and regular update of the suite.

Conclusion

Systematic coverage analysis and technique-based test design close the gaps and make system behaviour consistent and reliable.

Question 16: Improving Defect Reporting in a Logistics Application

Introduction

Poor reports cause delay because developers must ask for missing information before they can act. The solution is clear definitions and a standard format for reporting.

Clear Definitions Help Tracking

When the team uses the same meaning for error, defect, failure, severity and priority, each report is classified correctly, duplicates are avoided, statuses (new, assigned, fixed, retested, closed, reopened) are consistent, and metrics such as defect density can be trusted. Example: a wrong shipping cost shown on the screen is reported as a failure with actual and expected values, and the developer identifies the defect in the calculation code.

How Structured Reports Enhance Communication

A structured report contains defect ID, title, module, build/version, environment, steps to reproduce, expected result, actual result, severity, priority, attachments (screenshot, log), reporter and status. The developer can reproduce it without asking, the manager can prioritise it, and the tester can retest it with the same steps.

Example of a Good versus a Poor Report

Poor: "Tracking is not working." Good: "Shipment tracking shows 'Delivered' for ID LG1054, which is in transit. Steps: login, open Track Shipment, enter LG1054. Expected: In transit. Actual: Delivered. Environment: Chrome 120, Build 2.3. Severity: High."

Improvements in Reporting Standards

- Adopt a mandatory template and a defect tool such as Jira or Bugzilla with required fields.
- Define a severity and priority scale with examples, and set service-level times for response.

- Report each defect separately, objectively and with evidence; check for duplicates first.
- Review reports in regular defect triage meetings and give feedback to reporters.

Conclusion

Clear definitions and standard structured reports speed up identification, communication and resolution of defects.

Question 17: Process-based Testing for Consistency Across Modules

Introduction

Inconsistent testing arises when each module team follows its own method. Process-based testing defines a common set of phases, activities, templates, entry/exit criteria and roles that all modules follow.

Applying a Process-based Approach

- **Planning:** Common test plan template, scope, risks and schedule for each module.
- **Design:** Standard test case format and mandatory techniques, with traceability to requirements.
- **Execution:** Defined test environment, data, entry and exit criteria, and one defect life cycle.
- **Closure:** Test summary report, metrics and lessons learned.
- **Improvement:** Review the process using models such as TMMi or ISO/IEC/IEEE 29119.

How Standardised Processes Improve Quality

They make results comparable, reduce dependence on individuals, ensure no activity is skipped, speed up training of new members, allow reuse of assets, and make measurement and improvement possible. Defects are caught at the same phase in every module.

Example

In a university system, the Admission module team used checklists while the Examination module team tested informally, and the latter had more production defects. After a common process was adopted (same templates, review of test cases before execution, entry criteria of a stable build, exit criteria of no open critical defects), both modules were tested to the same depth and the defect leakage of the Examination module decreased.

Methods to Ensure Consistency

- Provide a documented test process, templates and checklists, and train all testers.
- Use a common test management and defect tool.
- Conduct peer reviews and process audits, and a QA/test lead to monitor adherence.

- Use common metrics (coverage, defect density, pass rate) and hold retrospectives.

Conclusion

A standardised, process-based approach gives consistent and predictable testing across all modules and improves overall quality.

Question 18: Edge Cases and Boundary Testing

Introduction

Edge cases are inputs or situations at the extreme limits of normal operation, such as minimum and maximum values, empty input and unusual combinations. Failures after deployment show that such cases were not covered.

Why Such Cases Were Missed

- Exhaustive testing is impossible and testing concentrated on typical inputs.
- Requirements did not state limits clearly and the testers did not question them.
- Test design relied on intuition and not on boundary techniques.
- Time pressure, and no analysis of defect history for similar defects.
- The pesticide paradox: the same tests were repeated.

How Boundary Testing Helps

Defects tend to appear at the edges of ranges due to errors in operators such as "<" instead of "<=". Boundary value analysis tests the values just below, at and just above each boundary, so these faults are found with few test cases.

Including Edge Cases in Testing

Example: a field accepting quantity from 1 to 100. Test 0, 1, 2, 99, 100 and 101, plus empty, negative, decimal, very large and non-numeric values. Other edge cases are the last day of a month or leap year (29 February), maximum file size, empty lists, zero balance, special characters, session timeout and simultaneous users. Identify them through requirement review, error guessing and past defects.

Improvements in Test Design

- Apply equivalence partitioning with boundary value analysis for every input field.
- Clarify limits in requirements and add them to acceptance criteria.
- Use checklists of common edge cases and review test cases with developers.
- Add exploratory and negative testing and automate boundary tests in regression.

Conclusion

Boundary and edge-case testing, supported by clear requirements and structured design, prevents post-deployment failures.

Question 19: Defect Documentation and a Structured Repository

Introduction

Without proper documentation, defects are forgotten, duplicated or mis-prioritised and no history exists for analysis. A defect repository with structured documentation solves this.

How Documentation Improves Defect Management

It allows tracking through the life cycle (new, assigned, open, fixed, retest, closed, reopened), assigns ownership, supports priority decisions, avoids duplicate effort, enables retesting using the same steps, and provides evidence and communication among testers, developers and managers.

How Structured Repositories Support Analysis

When each record has uniform fields (module, type, phase of origin, severity, priority, root cause, build and fix time), the data can be filtered and analysed for defect density, trends, defect age, defect distribution by module and cause, and rejection or reopen rates. Example: the repository shows that 50 percent of the open critical defects are in the Billing module, so effort is focused there.

Suggested Improvements in Documentation Practices

- Use a tool such as Jira, Bugzilla or Mantis as a single repository.
- Adopt a template with mandatory fields: ID, title, description, steps, expected and actual result, environment, severity, priority, attachments, owner and status.
- Write clear, objective, reproducible and unique reports, one per defect.
- Define classification and severity standards, and link defects to requirements and test cases.
- Update statuses promptly, record the root cause and the fix, and review the repository regularly.

Conclusion

Well-documented defects in a structured repository make tracking effective and give the data needed for analysis and prevention.

Question 20: Redesigning the Testing Approach for a Banking Transaction System

Introduction

Repeated failures in transaction processing show that testing was improper: unplanned, incomplete and not risk-based. Applying software testing principles, particularly early testing, risk-based testing, defect clustering and the impossibility of exhaustive testing, the approach is redesigned.

Redesigned Approach

- **Plan:** Test strategy and plan with risks, scope, entry/exit criteria and environment similar to production.
- **Early testing:** Review requirements for rules such as limits, charges and atomicity of debit and credit.
- **Design:** Create test cases using equivalence partitioning, boundary analysis, decision tables, state transition and use-case techniques, and trace them to requirements.
- **Execute and report:** Run tests, log defects in a tracker with a standard template and perform retesting and regression.
- **Measure and improve:** Use coverage, defect density and defect repository analysis to refine tests.

How Structured Testing Improves Quality

Critical flows are prioritised, boundary and failure conditions are covered, defects are found early and tracked to closure, and regression stops old failures from returning. Quality is measured and the results are repeatable.

Illustration with Examples

- Debit succeeds but credit fails due to a network error. Expected: the transaction is rolled back. Designed tests verify atomicity and consistency of balances.
- Boundary: transfer limit Rs. 1,00,000; test 99,999, 1,00,000 and 1,00,001.
- Concurrency: two simultaneous withdrawals from the same account must not overdraw it.
- Duplicate request: pressing Submit twice must debit only once.

Techniques to Enhance Defect Detection

- Black-box techniques with boundary and negative testing, and white-box coverage for the transaction logic.
- Concurrency, performance, stress and security testing on the transaction engine.
- Database verification through SQL for balance and ledger accuracy, with reconciliation tests.
- Automated regression (Selenium, REST Assured, JMeter), exploratory testing and root-cause analysis from the defect repository.

Conclusion

A planned, technique-based and measured testing approach reduces repeated transaction failures and increases the reliability and trustworthiness of the banking system.

Code of Conduct: *I P.Paneendra (192321072) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *P.Paneendra*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand